

UCS503 - Software Engineering

CivicSight

Smart Road Damage Detection and Municipal Repair Management System

Submitted by:

1024030154 Devansh Thapar

1024030156 Laksh Garg

1024030159 Pranay Mittal

Group: 3C-16 B.E 3rd Year, COE

Submitted to:

Ms. Paramveer Kaur

Prototype Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.2	Motivation	3
1.3	Problem Statement	3
1.4	Project Objectives	4
2	Methodology and Design	5
2.1	Overall Approach	5
2.2	System Architecture	6
2.2.1	High-Level Design	6
2.2.2	Request Flow when a Citizen Submits a Report	7
2.3	Technical Stack	8
2.4	UML Use Case Diagram	9
2.5	Lifecycle State Machine	10
2.6	ML Pipeline Design	11
3	Implementation	12
3.1	Backend	12
3.1.1	Authentication and Roles	12
3.1.2	Reports API	12
3.1.3	ML Service	12
3.2	Frontend	13
3.3	ML Subsystem	16
4	Results and Testing	18
4.1	Testing Strategy	18
4.2	Prototype Test Results	18
4.3	ML Results	19
4.4	Discussion	19
5	Conclusion and Future Work	20
5.1	Conclusion	20
5.2	Future Work	20
5.3	Lessons Learned	20

Introduction

1.1 Background and Context

Our roads take a lot of damage every day. Heavy vehicles, rain, heat, and old construction slowly create potholes and cracks. The problem is not just the damage itself — it is what happens after the damage appears. In most cities a citizen who spots a big pothole has to phone the municipality, maybe send a WhatsApp message, or visit an office. After that there is usually no way to know whether anyone actually saw the complaint, sent an inspector, or fixed the road. We wanted to build something that fixes this gap between “I saw a bad road” and “the road is repaired and I can see the proof.”

That is where CivicSight came from. CivicSight is a web-based platform where a citizen uploads a photo of road damage, the system figures out what kind of damage it is using an AI model, the report gets a priority, a municipal officer verifies it, a maintenance crew is assigned, and finally the ticket is closed — with every single step recorded so nothing is lost in between.

1.2 Motivation

During our Software Engineering course we had to pick a real problem and solve it the way a software company would — week by week, with a proper proposal, diagrams, code, and testing. Road damage reporting felt like a problem we had all seen in real life, so it was easy to stay motivated. All three of us worked across the frontend, backend, and ML parts together.

1.3 Problem Statement

The main problem we are solving is the missing link between road-damage reporting and actual repair. Today, reporting is scattered (calls, messages, walk-ins), assessment is subjective (a photo alone says nothing about severity), prioritization is manual, and there is no reliable audit trail of what happened to a complaint.

1.4 Project Objectives

1. Build a single web platform where citizens can report road damage with a photo, GPS location, and description — and get a trackable ticket ID. *(Done)*
2. Add an AI component that automatically detects damage type (pothole, cracks) in uploaded photos and shows the result with bounding boxes. *(Done)*
3. Give municipal officers a dashboard to verify, reject, mark duplicates, and assign repairs, with every change logged in a status history. *(Done)*
4. Enforce a strict server-side lifecycle so that, for example, a report cannot be assigned before it is verified. *(Done)*
5. Keep the citizen submission fast even though ML inference takes time, by running it in the background. *(Done)*

Methodology and Design

2.1 Overall Approach

We followed the iterative and incremental SDLC model throughout the project. Work happened in short, repeatable iterations (one per week), and in each iteration we designed, built, and tested a small vertical increment of the system. Because increments were kept small — reports CRUD in week 2, authentication and roles in week 3, map and ML baseline in week 4, and so on — we got early working software, received feedback from our own demos and tests, and fixed issues before they accumulated. The backend, frontend, and ML components were developed and integrated incrementally, and every increment was validated by its own test script before we moved on. Week 7 then tied these increments together into one integrated prototype. Week 1 was just the skeleton (folders, README, health checks). Week 2 added the citizen report form and the basic database. Week 3 brought in login and roles. Week 4 made it a real end-to-end flow with maps and the first ML experiment. Week 5 added the officer verification dashboard and an improved model. Week 6 made the lifecycle strict and finalized the ML module. Week 7 glued everything together into the prototype we are submitting.

Week	What we built
1	Monorepo skeleton (frontend, backend, ml folders), landing page, FastAPI health check, ML environment setup
2	Citizen report page with photo upload, backend models and CRUD APIs, RDD2022 dataset analysis
3	Registration/login with JWT, four roles (Citizen, Municipal Officer, Maintenance Staff, Admin), dataset splits and visual checks
4	Interactive Leaflet map with GPS, report creation API with image storage, preprocessing pipeline, first YOLOv8n baseline
5	Municipal officer dashboard (KPI cards, filters, inspection modal), verify endpoint, ML experiment 2
6	Full lifecycle state machine with strict transition rules, status history, reusable ML inference module
7	End-to-end integration: async ML processing on upload, four ML states, six-checkpoint prototype audit

2.2 System Architecture

2.2.1 High-Level Design

CivicSight has three parts: the **frontend** (plain HTML/CSS/JavaScript pages in the browser), the **backend** (a FastAPI server that owns all data and business rules), and the **ML subsystem** (Python code using PyTorch and Ultralytics YOLOv8). The browser never talks directly to the ML code — everything goes through the backend, which is important because it means we can check permissions and log every action in one place.

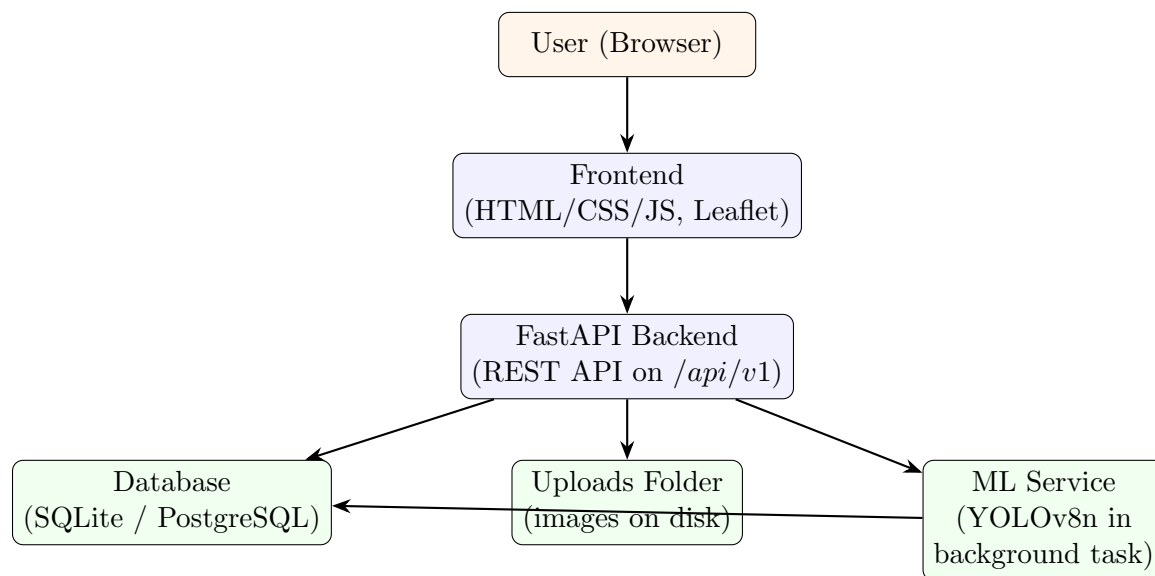


Figure 2.1: High-level architecture of CivicSight.

2.2.2 Request Flow when a Citizen Submits a Report

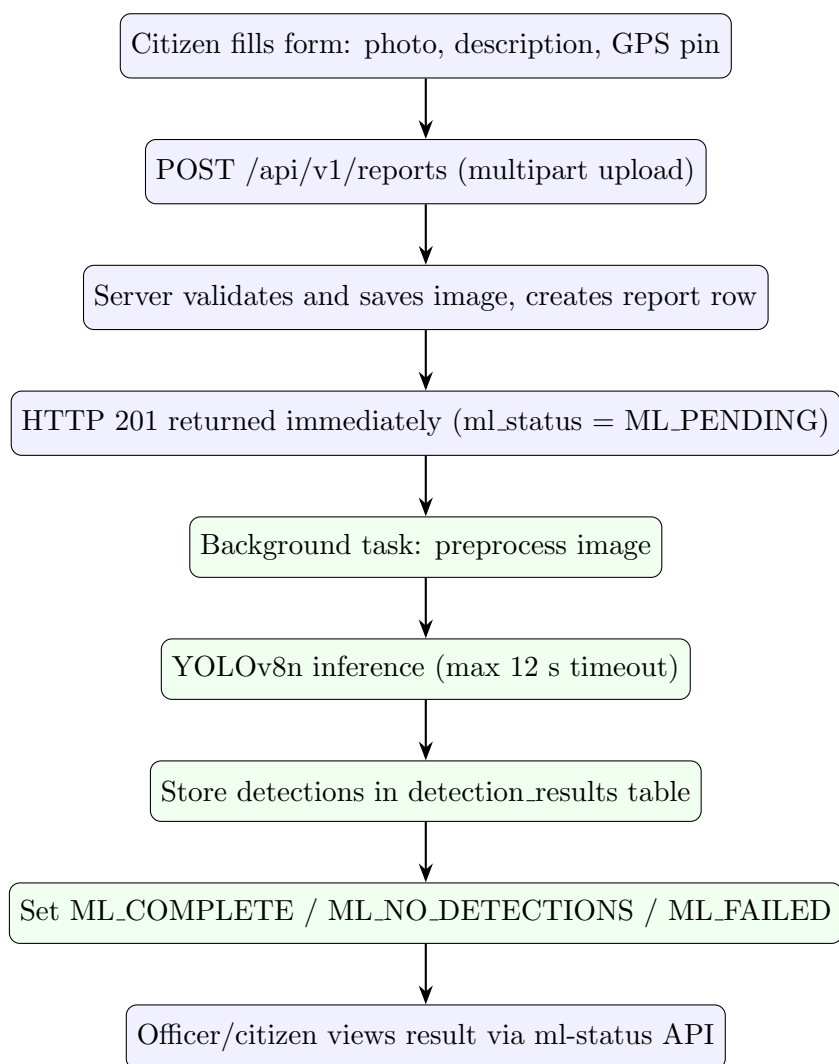


Figure 2.2: Report submission and asynchronous ML processing flow.

The key design decision was that steps (e) through (h) happen *after* the HTTP response. In our tests a submission returns in about 12 ms, while the AI work takes about 40–50 ms plus overhead — so nothing ever feels slow for the citizen.

2.3 Technical Stack

- **Backend:** Python 3.10+, FastAPI, SQLAlchemy 2.0, PyJWT, bcrypt, Uvicorn.
- **Database:** SQLite for local development; PostgreSQL for real deployment.
- **Frontend:** plain HTML, CSS, and JavaScript (no framework), Leaflet.js for the map, Lottie animation for the scanning effect.
- **ML:** PyTorch, Ultralytics YOLOv8n, dataset: RDD2022 with classes D00 (longitudinal crack), D10 (transverse crack), D20 (alligator crack), D40 (pothole).
- **Testing:** small Python test scripts that run against a live server.

2.4 UML Use Case Diagram

Figure 2.3 shows everyone who uses CivicSight and what each of them can do.

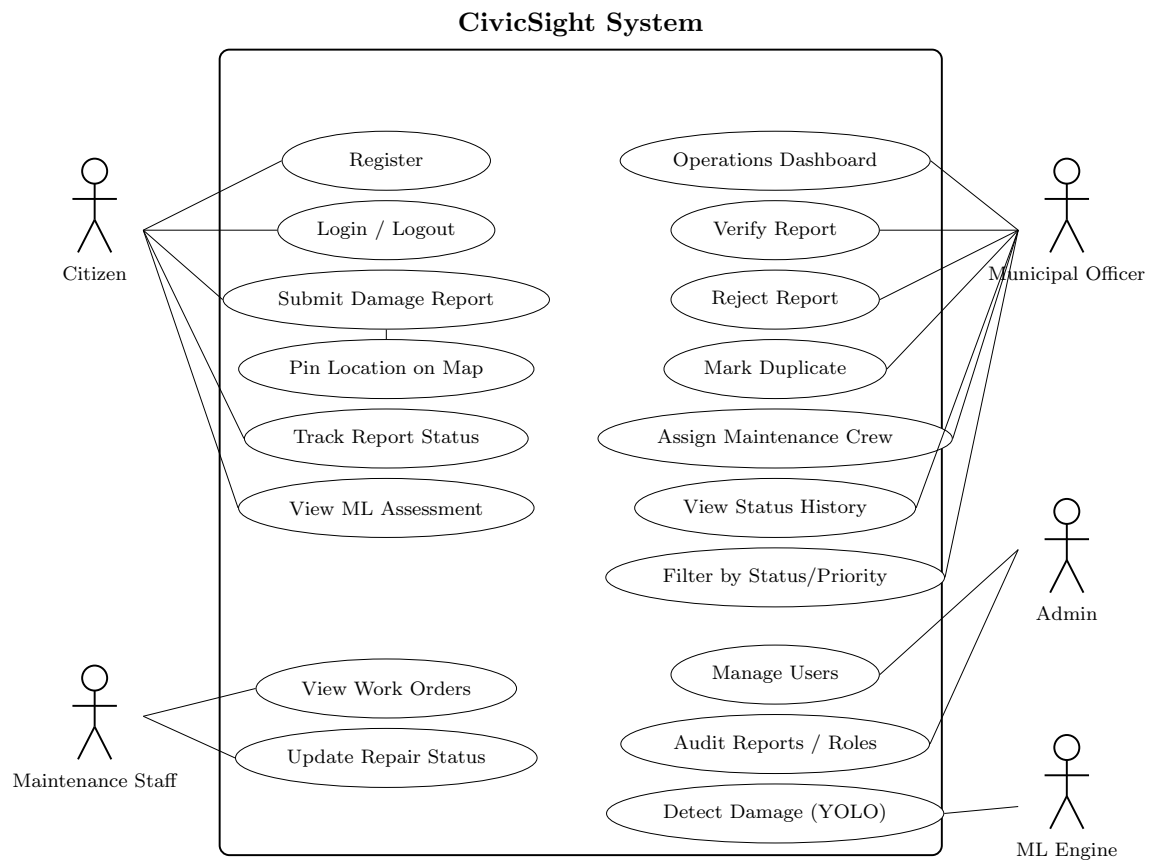


Figure 2.3: Use case diagram covering all CivicSight functionalities.

2.5 Lifecycle State Machine

A report in CivicSight can be in these states: `submitted`, `pending_verification`, `verified`, `rejected`, `duplicate`, `assigned`, `under_repair`, `repaired`, `closed`. The important part is that the server itself decides which moves are legal — the frontend cannot cheat the workflow.

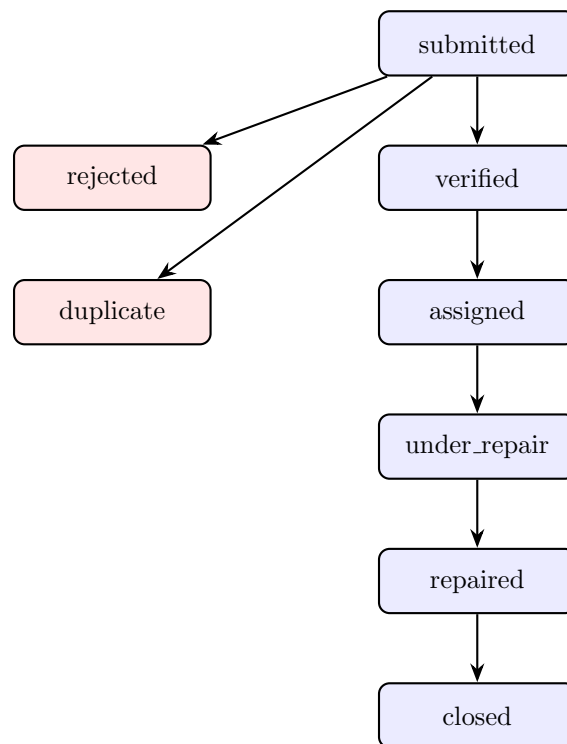


Figure 2.4: Legal lifecycle transitions of a report (simplified).

Some rules the server enforces: a report cannot be assigned before it is verified; once assigned it cannot be rejected; a closed or repaired report cannot be verified again. Breaking the rules returns an HTTP 400 error. Every allowed change is written into a `report_status_history` table, which gives the officer a full audit trail.

2.6 ML Pipeline Design

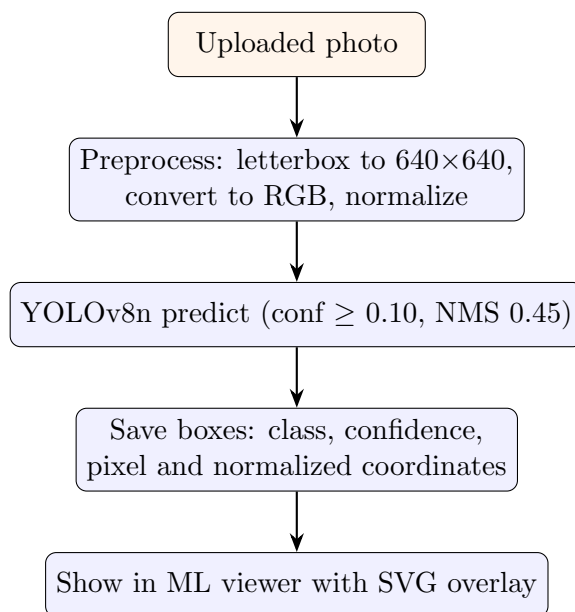


Figure 2.5: Machine learning pipeline inside the backend background task.

We used the RDD2022 dataset, split as 26,869 training, 5,758 validation, and 5,758 test images. We trained two experiments. Experiment 1 (Week 4) was very weak (mAP@0.5 of 0.0335). Experiment 2 (Week 5) fixed this by lowering the input size to 512, training for 3 epochs, and using cosine learning-rate scheduling — precision jumped from 0.0041 to 0.6541 and mAP@0.5 went up by about 251%.

Implementation

3.1 Backend

The backend is built with FastAPI. The main file `app/main.py` starts the app, enables CORS (so the frontend on another port can call it), mounts the `/uploads` folder for serving images, and registers the API routers. All database models live in `app/models/models.py`.

3.1.1 Authentication and Roles

Registration stores a bcrypt hash of the password — never the plain password. Login checks the password against the hash and returns a JWT token that the frontend keeps in `localStorage`. Two reusable helper functions protect endpoints: one that checks “is this a logged-in user”, and one that checks the role (e.g. only Municipal Officer and Admin). We tested that a citizen token gets 403 Forbidden on officer endpoints.

3.1.2 Reports API

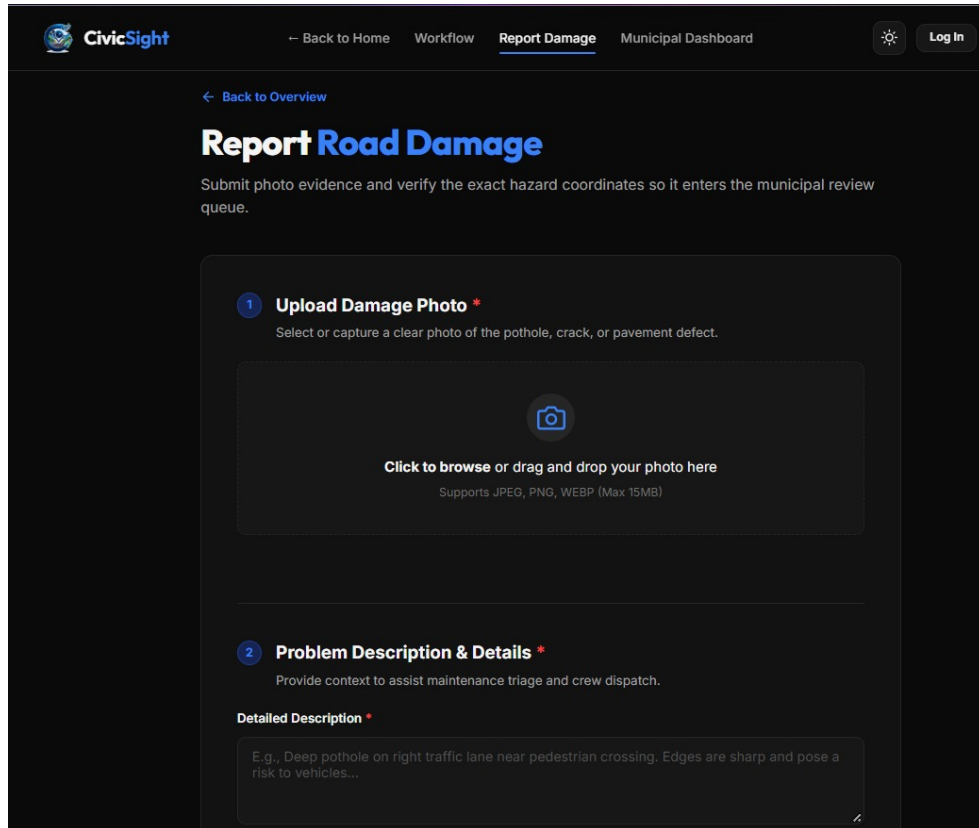
Endpoint	Purpose
POST <code>/api/v1/auth/register</code>	Create a new account
POST <code>/api/v1/auth/login</code>	Get a JWT token
POST <code>/api/v1/reports</code>	Submit a report with photo and location
GET <code>/api/v1/reports</code>	List reports, filter by status/priority
GET <code>/api/v1/reports/{id}/ml-status</code>	Check ML processing state
PATCH <code>/api/v1/reports/{id}/verify</code>	Officer verifies the report
PATCH <code>/api/v1/reports/{id}/assign</code>	Give to a maintenance crew
GET <code>/api/v1/reports/{id}/history</code>	Full audit timeline

3.1.3 ML Service

The file `app/services/ml.service.py` runs inside a background task. It takes the saved image, runs preprocessing and inference with a timeout, writes one row per detected defect into `detection_results`, and updates the report’s `ml_status`. Because the UI has four possible ML states (pending, complete, no detections, failed), the frontend always knows exactly what to show — it never displays a confusing blank box.

3.2 Frontend

The frontend has no framework — just static pages. We keep it simple so that hosting is trivial (any static server works).



The screenshot shows the 'Report Road Damage' page in the CivicSight application. The page has a dark theme. At the top, there is a navigation bar with the CivicSight logo, a 'Back to Home' link, a 'Workflow' link, the 'Report Damage' link (which is active), and a 'Municipal Dashboard' link. On the right side of the navigation bar, there are a settings icon and a 'Log In' button. Below the navigation bar, there is a 'Back to Overview' link. The main heading is 'Report Road Damage' in a large, bold, blue font. Below the heading, there is a subheading: 'Submit photo evidence and verify the exact hazard coordinates so it enters the municipal review queue.' The form is divided into two main sections. The first section is '1 Upload Damage Photo *'. It includes a subtext: 'Select or capture a clear photo of the pothole, crack, or pavement defect.' Below this, there is a large, light gray rectangular area with a camera icon in the center. The text 'Click to browse or drag and drop your photo here' is centered in this area, with 'Supports JPEG, PNG, WEBP (Max 15MB)' written below it. The second section is '2 Problem Description & Details *'. It includes a subtext: 'Provide context to assist maintenance triage and crew dispatch.' Below this, there is a 'Detailed Description *' label. Underneath, there is a text input field with a placeholder text: 'E.g., Deep pothole on right traffic lane near pedestrian crossing. Edges are sharp and pose a risk to vehicles...'. At the bottom right of the form, there is a small icon of a map.

Figure 3.1: Damage report page: drag-and-drop photo, description, and Leaflet map with GPS.

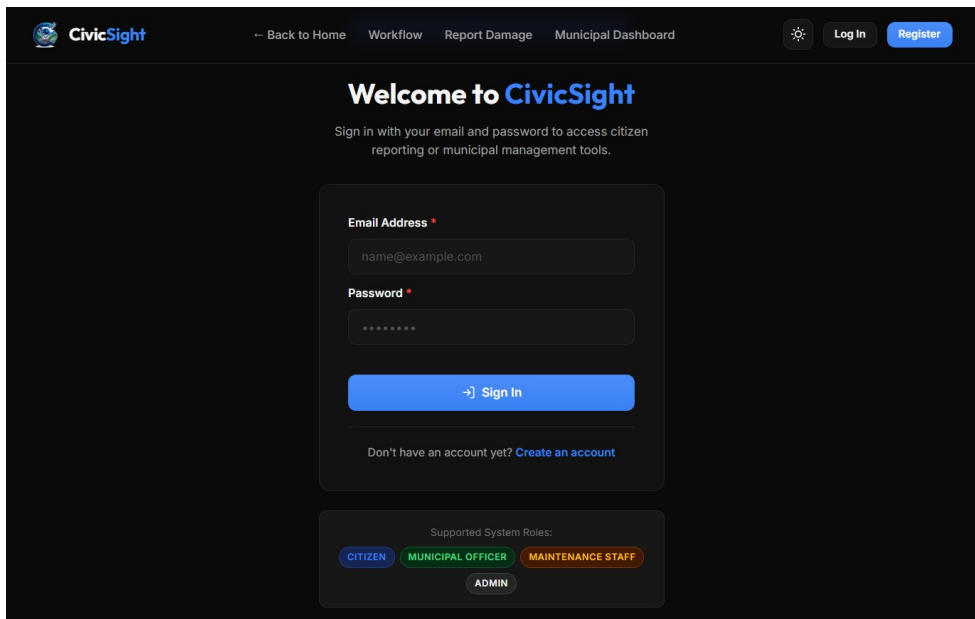


Figure 3.2: Municipal officer dashboard with KPI cards, filters, and live report table.

• AI ASSESSMENT (DECISION SUPPORT)

ML_COMPLETE

HIGH PRIORITY (AI ESTIMATE)

Defect Analysis Preview

Decision Support Only: AI detections and severity estimates assist municipal triage and do NOT constitute official verification.

Model: YOLOv8n-experiment2_week5



AI Estimated Impact Rating 95% - High Priority Remediation

CLASSIFIED DEFECTS (1)

D40

Pothole Hazard
YOLOv8n-experiment2_week5

89%
CONFIDENCE

GEOSPATIAL COORDINATES

Latitude:

37.774900

Longitude:

-122.419400

Address:

Field Pinpoint

Figure 3.3: Inspection modal showing the AI detection with bounding boxes and the audit timeline.

3.3 ML Subsystem

We keep all ML code inside the `ml/` folder: `data.yaml` points to the dataset, `src/preprocess.py` contains the reusable preprocessing, `src/inference.py` is the reusable inference function used by the backend, and `scripts/` has the scripts for training, testing, and dataset visualization. The weights used in production are from experiment 2, saved at `ml/runs/detect/experiment2_week5/weights/best.pt`.

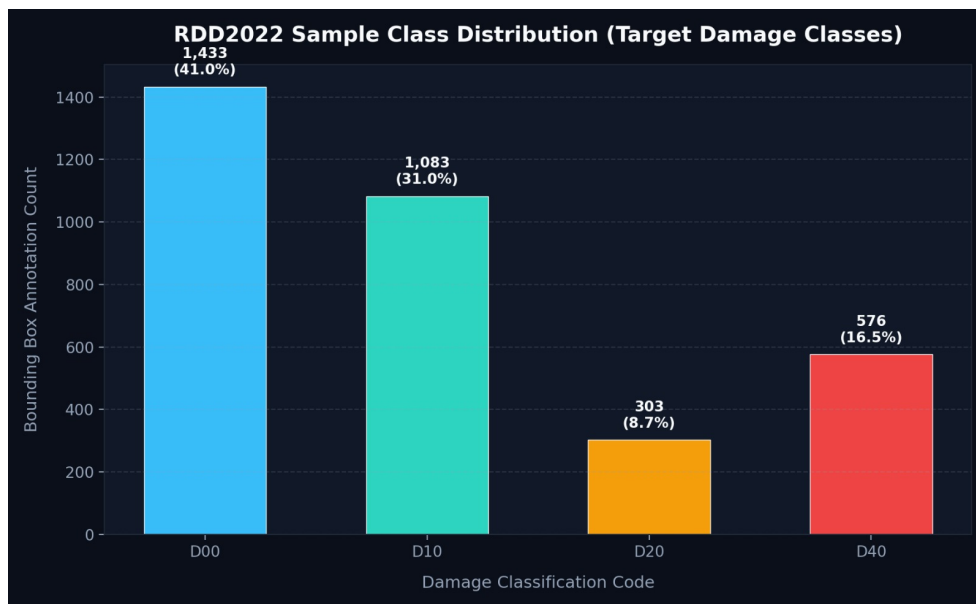


Figure 3.4: Distribution of classes in the RDD2022 training split.

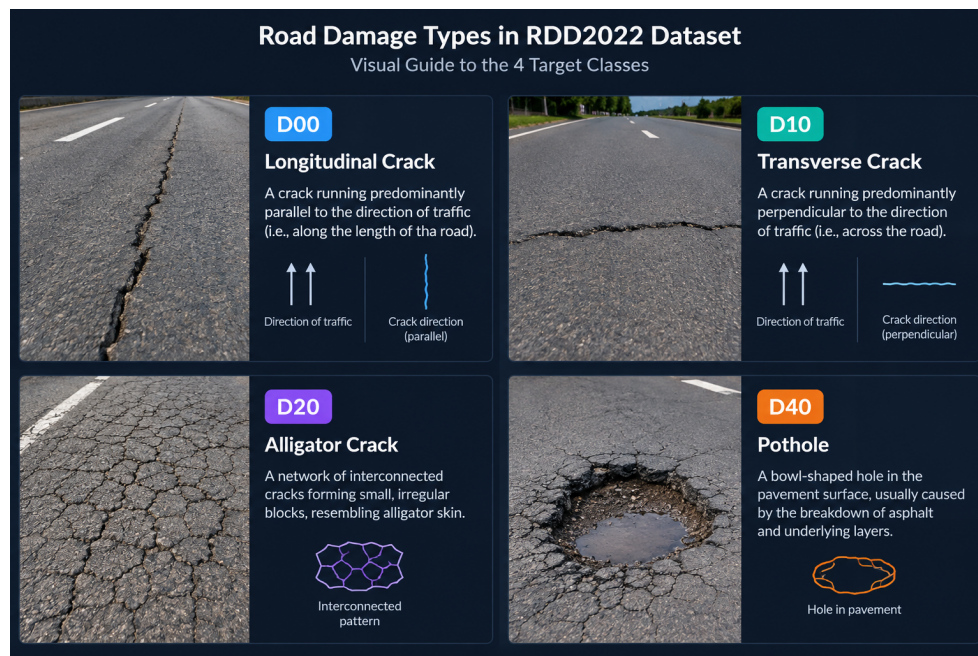


Figure 3.5: Sample YOLOv8n detection output with bounding boxes on a road image.

Results and Testing

4.1 Testing Strategy

We did not follow heavy formal testing — we kept it practical:

- Small Python test scripts for authentication (register, login, token checks, RBAC 403s).
- Test scripts for the reports API: creating reports, filtering, verification rules.
- Test scripts for the lifecycle: valid path works, illegal paths return 400, history rows appear.
- A six-checkpoint end-to-end script (Week 7) that runs against a live server and a real database.

4.2 Prototype Test Results

#	What we tested	What happened	Result
1	Upload a real pothole photo	Report created in 11.6 ms, ml.status = ML_PENDING	Pass
2	Wait for ML to finish	ML_COMPLETE, D40 pothole detected, confidence 0.335	Pass
3	Upload a clean road photo	ML_NO_DETECTIONS, no error	Pass
4	Upload a corrupted image	ML_FAILED, but the report itself stayed valid	Pass
5	Officer verifies the report	Status changed to verified; AI section stayed separate	Pass
6	Restart the server	Same detections still returned by the API	Pass

4.3 ML Results

Metric	Exp 1	Exp 2	Change
Precision	0.0041	0.6541	+0.65
Recall	0.5024	0.1254	−0.377
mAP@0.5	0.0335	0.1176	+251%
mAP@0.5:0.95	0.0114	0.0530	+365%

Table 4.2: YOLO experiment comparison.

Potholes (D40) are the easiest class for our model; alligator cracks (D20) are still very hard and that is our biggest weakness. Latency on a normal laptop CPU: preprocessing about 8ms, inference about 40ms, total pipeline under 50ms.

4.4 Discussion

The prototype does what we aimed for: a complete path from a citizen complaint to a closed ticket, with an AI assist in the middle. Two honest observations: (1) the model’s recall is low — it prefers to stay quiet rather than shout wrong answers, which we think is the right trade-off for a first version; (2) the async ML design was the single most important decision in the project, because it made the whole system feel fast regardless of model speed.

Conclusion and Future Work

5.1 Conclusion

In seven weeks we took CivicSight from an empty folder to a working prototype. A citizen can register, report a road problem with a photo and a map pin, and watch the ticket get detected, verified, and repaired — while the municipality gets a dashboard with real numbers and a full audit trail. The AI part is honest about its limits: it shows its confidence and never pretends to be the final word.

5.2 Future Work

- Proper notifications (email/SMS) when a ticket changes status.
- Improve D20 alligator-crack detection with more data and longer training.
- A map view showing all reports together, so hotspots are visible.
- Deploy on a real server with PostgreSQL and HTTPS.
- Mobile-friendly PWA so citizens can report directly from the spot.

5.3 Lessons Learned

Keeping the frontend, backend, and ML in one monorepo saved us a lot of confusion. Writing small test scripts early (instead of testing only at the end) caught many bugs — especially illegal lifecycle moves. And planning the async ML design on paper first made Week 7 much less painful than it would have been otherwise.